SaTTric+ Configuration UI

A Developer's Guide to Building the Setup & Rules Interface

No electronics or embedded-systems background required.

This guide explains, in plain language, what the SaTTric+ device needs from its configuration UI, what data the UI collects, and how that data is sent to the device. Every technical term is explained the first time it appears.

Contents

1. What SaTTric+ Is (in plain terms)

SaTTric+ is a small industrial box that gets installed in a factory, building, or plant. It plugs into things like electricity meters, temperature sensors, pressure sensors, water/gas meters, and switches (pumps, fans, alarms). It reads all of that, decides if anything needs attention, and can turn things on or off automatically.

The important thing for you as a UI developer: SaTTric+ does not come pre-programmed for one specific meter or one specific sensor. Every installation is different — one customer has a Schneider electricity meter, another has a Secure meter; one site has a 4-20mA pressure sensor, another has a simple on/off door switch. Instead of shipping different firmware for every combination, SaTTric+ is designed to be told what is connected to it, in the form of configuration data sent from your UI.

> *Your job is not to understand electrical engineering. Your job is to build forms that collect the right pieces of information from the person installing the device (an electrician or technician), and turn that into a data package the device understands.*

2. The Big Idea: Everything Is a Form That Becomes Data

Think of SaTTric+ like a very configurable smart-home hub, except for industrial equipment. There are four things a technician needs to describe when setting up a device:

- Which electricity meter is connected, and how to read its measurements (Section 4)
- What is plugged into each of the 8 general-purpose input channels — a sensor or a switch (Section 5)
- What each output (a relay or a variable-speed control) should be connected to, and how it should behave (Section 6)
- Which communication ports are being used for what, e.g. a second meter bus, or a cellular modem (Section 7)

On top of these four setup areas sits the Rule Engine (Section 8) — this is where the technician or facility manager defines "if this happens, do that" logic, such as "if the tank level drops below 20%, turn on the booster pump."

Every one of these five things is represented as a JSON object in your UI, and gets sent to the device (through a cloud/API layer — see Section 9) as a configuration update. The device stores it, and immediately starts behaving according to the new configuration. There is never a firmware reflash involved in normal use.

2.1 A Few Words You Will See Throughout This Guide

Term	Plain-English meaning
Modbus	A common language that industrial meters and devices use to report their readings. Think of it as "the API format that hardware speaks."
Register	A single numbered "slot" of data inside a Modbus meter — e.g. register 40001 might hold the voltage reading. Every meter brand numbers its slots differently.
Register map	A lookup table that says "register X means voltage, register Y means current" for one specific meter model. This is exactly what your UI needs to help the technician build.
Channel	One physical input or output wire terminal on the device, e.g. "Input Channel 3."
Analog signal	A reading that varies smoothly, like a voltage between 0-10V representing a pressure between 0-10 bar.

Term	Plain-English meaning
Digital signal	A simple on/off, yes/no signal — e.g. a door switch that is either open or closed.
Calibration	Telling the device "when the raw signal reads X, that actually means Y in real units" — e.g. "0V = 0 bar, 10V = 16 bar."
Relay	An electrically controlled switch the device can turn on/off to control something like a pump or an alarm light.
PWM output	A variable output (not just on/off) used for things like speed or valve position control.
Tag / Tag Database	Every live reading (voltage, temperature, pump status, etc.) is stored under a short internal name called a "tag," e.g. tag L1_Voltage. Rules and dashboards refer to data by its tag name.

3. How This Fits Together

You will typically be building against a cloud backend service, not talking to the physical device directly. The recommended flow is:

- 1. Your UI collects setup information through forms (meter brand, sensor types, thresholds, etc.)
- 2. Your UI sends this as JSON to a backend API (e.g. POST /devices/{id}/config/meter)
- 3. The backend translates the JSON into the compact binary format the physical device needs, and delivers it (over the internet or a local connection)
- 4. The device stores the new configuration as a "draft" and only activates it once a "commit" / "deploy" step is confirmed — similar to Save Draft vs. Publish
- 5. The device reports back success/failure, and your UI shows the technician a confirmation

> *You never need to construct the low-level binary protocol yourself. That translation layer is a backend concern. This guide focuses on the JSON shapes your UI needs to produce and consume.*

4. Screen: Energy Meter Setup

Purpose: let the technician tell the system which electricity meter is wired to SaTTric+, and how to read its data.

4.1 What the technician sees

In practice, most technicians will not fill this in from scratch. Your UI should ship with a library of pre-built templates for common meter brands (Schneider, Secure, Rishabh, etc.) — the technician just picks a brand and model from a dropdown, and the register map is already filled in behind the scenes. The manual form below is only needed for a meter model that isn't in the library yet.

First choice: how is the meter physically connected? SaTTric+ supports both a serial connection (RS-485, the traditional way most industrial meters connect) and an Ethernet connection (for newer meters that expose their data over the plant network). Ask for this up front, since the fields below it change slightly depending on the answer.

Form field	Example value	Notes for the UI
Connection type	RS-485 (serial)	Two options: "RS-485 (serial)" or "Ethernet (network)"
Meter brand / model	Schneider iEM3255	Dropdown, backed by your template library
— if RS-485 — Modbus address (slave ID)	1	A number 1-247, set on the meter itself by the electrician
— if RS-485 — Communication speed (baud rate)	9600	Dropdown: 9600 / 19200 / 38400 etc.
— if RS-485 — Parity / stop bits	None / 1	Dropdown, rarely changed from defaults
— if Ethernet — Meter IP address	192.168.1.50	The meter's network address, set during its own network configuration
— if Ethernet — Port	502	Almost always 502, the standard Modbus-over-Ethernet port; pre-fill it
— if Ethernet — Unit ID	1	Plays the same role as the slave ID does for RS-485
Which measurements to read	Voltage L1, Current L1, Total kWh, Power Factor…	Checklist — see the 114-parameter list already defined for SaTTric+

> *The measurement checklist and everything in Section 4.2 below is identical regardless of which connection type is chosen — only the handful of connection-specific fields above change. Your template library entries should work for either connection type without needing two separate templates per meter model.*

4.2 What gets sent to the device (per measurement)

For every measurement the technician selects (e.g. "Voltage L1"), the template supplies these details automatically — your UI does not need to ask the technician for them if a template is used:

```
{
  "parameter": "Voltage_L1",
  "modbus_function": "READ_HOLDING_REGISTERS",
  "start_register": 3027,
  "register_count": 2,
  "data_type": "FLOAT32",
  "byte_order": "BIG_ENDIAN",
  "scale_factor": 1.0,
  "offset": 0.0,
  "poll_interval_ms": 2000
}
```

If a technician is entering a brand-new, unlisted meter, this is exactly the information they'd copy out of that meter's own manual (every meter datasheet publishes its register list) — your form should mirror these same fields, with plain labels: "Where is this reading stored" (start_register), "How many registers does it take" (register_count), "What format is the number in" (data_type).

5. Screen: Sensor / Input Setup

Purpose: SaTTric+ has 8 general-purpose input terminals. Each one can be wired to a different kind of sensor or switch. The UI needs to let the technician describe, for each of the 8 channels, what's plugged in.

5.1 Step 1 — What kind of signal is this?

Choice	Plain description	Typical use
0–10V analog	A smooth voltage signal	Pressure, level, or position transmitters
4–20mA analog	A smooth current-loop signal, more noise-resistant over long cable runs	Industrial pressure/temperature transmitters
Digital (on/off)	A simple switch or contact, 10–30V	Pump run status, door contact, alarm contact
Pulse counting	Counts pulses over time (device has 3 dedicated pulse channels, separate from the 8 general channels)	Legacy water, gas, steam, or fuel meters

5.1b For digital channels only — which way is "on"?

A door switch wired normally-open and one wired normally-closed report opposite raw signals for the same real-world state. Ask a simple either/or question so the tag value always means the same thing regardless of how the field wiring happens to run:

Question asked in the UI	Example answer
When this contact is closed, does that mean the monitored condition is active?	Yes — "closed = door open" (a normally-open door switch)

Phrase it in terms of the real-world condition ("door open", "pump running"), not electrical jargon ("active-high") — the technician wiring the panel knows which way their switch is oriented, but may not immediately map that to a polarity term.

5.2 Step 2 — Real-world scaling (for analog channels only)

This is the part that most needs a friendly UI, because it's the one place a non-technical installer could get confused. The idea: the sensor sends a raw electrical signal, but the person configuring the system thinks in real units (bar, °C, %). Your form should ask for two reference points and calculate the rest automatically — this is standard practice called "two-point calibration."

Question asked in the UI	Example answer
What does this sensor read at its lowest raw signal (e.g. 0V or 4mA)?	0 bar
What does this sensor read at its highest raw signal (e.g. 10V or 20mA)?	16 bar
What unit is this measured in?	bar

A good UI pattern: show a simple two-row table — "Low point" and "High point" — each with a raw value and a real-world value, exactly like calibrating a scale. The device then does straight-line math between those two points for every reading.

5.3 What gets sent to the device

```
{
  "channel_id": 3,
  "mode": "ANALOG_4_20MA",
  "tag_name": "ChilledWater_Pressure",
  "unit": "bar",
  "calibration": {
    "raw_low": 4.0, "value_low": 0,
    "raw_high": 20.0, "value_high": 16
  },
  "smoothing_ms": 500
}
```

6. Screen: Output Setup

Purpose: SaTTric+ can control up to 8 on/off outputs (relays) and 4 variable outputs (PWM). This screen lets the technician describe what each output is wired to, and which of five control authorities is allowed to drive it.

Form field	Example value	Notes
Output	Relay 2	Dropdown of the 8 relays + 4 variable outputs
What is this wired to?	Booster pump contactor	Free-text label, shown throughout the UI so users don't have to remember "Relay 2"
Control mode	Rule-driven	Manual / Rule-driven / Scheduled / Remote-supervisory / AI-driven — see 6.1
If rule-driven, which rule?	"Low tank level → start booster pump"	Link to a rule built in Section 8
If remote or AI-driven, watchdog timeout	30 seconds	How long the device waits without a command before falling back to a safe state
Safe state on fault	OFF	What the output should do if communication is lost — this matters for safety

> *Always ask for a "safe state on fault" for every output the technician configures. If the device loses its connection or crashes, this is what keeps a pump or valve from being left in a dangerous state.*

6.1 Five control modes, not three

This device recognizes five separate control authorities for an output, and only one can be active at a time:

Mode	Who's actually in control	When to use it
Manual	A person clicks a button, once	Commissioning, manual override
Rule-driven	The device's own local rule engine	Safety interlocks — works with no network at all
Scheduled	A stored time schedule	Time-of-day load shedding
Remote-supervisory	An authenticated person, acting through this UI	Deliberate operator override from the dashboard
AI-driven	The server's AI/ML optimization loop	Automated optimization, demand response

The reason "Remote-supervisory" and "AI-driven" are two separate modes rather than one "Remote" mode: every command your UI sends to one of these outputs is tagged, on the device itself, with who sent it — a person's user ID or the AI model's session ID. That tag becomes part of the device's own audit trail (Section

8.5), which is what lets an auditor later answer "did a person do this, or did the algorithm?" from the device's own record, not just from server logs that sit upstream of the device and could be incomplete.

> *An output set to Rule-driven will reject a live command from your UI, even from an authenticated operator — that's intentional. Changing who's allowed to control a safety-relevant output is a deliberate configuration change (switch its control mode, then deploy), not something that should be possible via a one-off button click.*

7. Screen: Communication Port Setup

Purpose: SaTTric+ has a few physical communication ports that can each be assigned a different job. Most installations won't need to touch this screen — it exists for advanced setups (a second meter bus, a non-Modbus sensor, or a cellular modem swap).

Port	Typical assignment	When a technician would change it
Meter bus	Primary electricity meter (Section 4)	Rarely changed
Auxiliary bus	A second Modbus meter or device	Site has more than one meter to read
Cellular/GPRS port	Sends data to the cloud	Swapping to a different SIM/modem module
Diagnostic port	Factory setup and troubleshooting	Field technicians only, not end users

Each port's form just needs speed (baud rate), and a mode dropdown ("Modbus meter", "Raw sensor data", "Diagnostic"). Nothing here is different in kind from the meter setup screen — it's the same idea of describing communication settings, applied to a different physical wire.

8. Screen: Rule Builder ("If This, Then That")

This is usually the screen facility managers spend the most time in, and it's worth designing carefully even though the underlying idea is simple: watch one value, compare it to a threshold, and trigger an action when the comparison is true.

8.1 The four things every rule needs

Piece	Plain description	Example
Watch	Which reading to monitor	Current_L1 (from the meter) or ChilledWater_Pressure (from a sensor)
Compare	Is it greater than, less than, equal to…	greater than
Threshold	The number to compare against	100 (Amps)
Action	What to do when true	Turn ON Relay 1 (Alarm horn)

8.2 Hysteresis — preventing a rule from "flickering"

If a reading sits right at the threshold (e.g. current bouncing between 99A and 101A), a rule with no buffer would switch its output on and off rapidly, which can wear out relays and confuse operators. The fix is a second, smaller number called hysteresis — the value has to fall a bit below the threshold before the rule turns back off. Your rule builder should always ask for this, with a sensible pre-filled default, rather than leaving it to a raw text field.

In plain terms for the UI copy: "Turn on when Current L1 goes above 100A. Turn off again once it drops below 95A." (100A is the threshold, 5A is the hysteresis.)

8.3 Realistic example rules

These examples are grounded in the actual sensors and assets SaTTric+ is designed to work with (from the site's monitoring parameter list and asset matrix), and are good candidates for a "starter rule" template library in your UI:

Rule name	Watch (tag)	Condition	Action	Why it matters
High feeder current	Current_L1	> 100 A (hyst. 5A)	Turn ON Relay 1 — alarm horn	Early warning before a breaker trips
Low booster tank level	Tank_Level_Pct	< 20 % (hyst. 5%)	Turn ON Relay 2 — booster pump	Keeps water pressure available
High chiller discharge pressure	Discharge_Pressure	> 18 bar (hyst. 1 bar)	Turn OFF Relay 3 — compressor; Turn ON Relay 4 — alarm	Protects compressor from damage
Poor power factor	Total_Power_Factor	< 0.85 (hyst. 0.02)	Send notification only (no output)	Flags avoidable utility penalty
UPS running on battery	UPS_OnBattery_Status	== TRUE	Send notification, log event	Distinguishes backup power events from normal

Rule name	Watch (tag)	Condition	Action	Why it matters
				use
After-hours lighting left on	Lighting_Circuit_Status	== TRUE (during "after-hours" schedule)	Turn OFF Relay 5 — lighting contactor	Cuts avoidable non-production energy use
Filter needs attention	Filter_Dirty_Alarm	== TRUE	Send maintenance notification	Prevents rising fan energy from a choked filter
Cold room too warm	ColdRoom_Temperature	> 6 °C for 10 min	Send notification, log event	Protects stored product, flags door-discipline issues

> *A great UX shortcut: ship these as one-click "starter rules" the user can drop onto their site and just adjust the threshold number, rather than making everyone build every rule from a blank form.*

8.4 What a rule looks like as data

```
{
  "rule_name": "High feeder current",
  "watch_tag": "Current_L1",
  "operator": "GREATER_THAN",
  "threshold": 100,
  "hysteresis": 5,
  "action": {
    "output": "Relay_1",
    "type": "SET_ON"
  },
  "also_notify": true
}
```

Multi-condition rules ("if A AND B") and time-based conditions ("only after 6pm") are natural extensions of this same shape — represent them as an array of conditions joined by AND/OR, and an optional schedule window, rather than inventing a separate rule type for each.

8.5 Two screens that exist specifically for audits

Two more pieces of this device exist because the product is positioned around auditability, not just optimization. Both are read-mostly screens — the technician rarely writes to either day-to-day.

Screen	What it shows	Who uses it
Audit log viewer	A chronological, filterable list of every output change, config deployment, calibration edit, and carbon-factor update — each entry shows exactly who made it (a named user, the AI, or the local rule engine) and when	Auditors, compliance reviewers, and support staff investigating an incident
Carbon factor profiles	The active kg-CO2e-per-kWh conversion factor(s) for the site, each with a validity window (start/end date)	Sustainability/ESG staff who need to update the

Screen	What it shows	Who uses it
	and a region code	grid factor when it changes

The audit log is generated entirely by the device itself — your UI's job is to display it, filter it (by date range, by output, by actor type), and let someone export it, not to construct entries. This is a deliberate design choice upstream of the UI: the record of "who did what" needs to be trustworthy even if a server-side log were ever incomplete, tampered with, or simply never received an update — which is exactly the failure mode an audit is meant to catch.

```
{
  "timestamp": "2026-08-15T14:32:01Z",
  "actor_type": "REMOTE_SUPERVISORY",
  "actor_id": "user_4821",
  "event_type": "OUTPUT_STATE_CHANGE",
  "target": "Relay_2",
  "old_value": 0,
  "new_value": 1
}
```

Carbon factor profiles follow the same stage-then-deploy pattern as everything else in this guide: a form to set the factor value and validity window, saved as a draft, then published — and every published change is automatically what shows up in the audit log as a CARBON_FACTOR_UPDATE entry.

9. Talking to the Device: A Simple Example

Below is the kind of request/response your UI will realistically make against the backend API — this is illustrative of the pattern (your backend team will confirm exact endpoint names), not a byte-level specification.

Saving a draft configuration

```
POST /devices/SAT-2201/config/rules
Content-Type: application/json

{ "rule_name": "High feeder current", "watch_tag": "Current_L1",
  "operator": "GREATER_THAN", "threshold": 100, "hysteresis": 5,
  "action": { "output": "Relay_1", "type": "SET_ON" } }

Response: 202 Accepted
{ "status": "staged", "config_version_draft": 14 }
```

Publishing (deploying) the staged changes

```
POST /devices/SAT-2201/config/commit

Response: 200 OK
{ "status": "active", "config_version": 14 }
```

Design your UI around this two-step "stage, then commit" pattern — let the technician make several changes across multiple screens, review a summary, and publish everything together. This avoids the device ever running on a half-finished configuration, and gives you a natural place to show a "Review changes before deploying" confirmation screen.

10. Suggested Screen List

Screen	Purpose	Key elements
Device setup wizard	First-time onboarding for a new device	Site name, device serial, network/SIM setup
Meter library & setup	Section 4	Brand/model picker, manual register entry fallback, RTU or Ethernet transport choice
Input channel mapping	Section 5	8-channel grid, per-channel signal type + calibration + polarity
Output mapping	Section 6	8 relays + 4 variable outputs, label + control mode (5 options) + watchdog timeout + safe state
Communication ports	Section 7	Advanced/collapsed by default
Rule builder	Section 8	Starter rule templates, watch/condition/action form, hysteresis field
Audit log viewer	Section 8.5	Chronological, filterable by actor/date/output; export
Carbon factor profiles	Section 8.5	Factor value, validity window, region code
Review & deploy	Section 9	Diff-style summary of staged changes, single Publish button
Live dashboard	Ongoing monitoring	Current tag values, active alarms, rule history, device health

Appendix A — Field Reference (for developers who want the full detail)

This appendix mirrors the underlying protocol structures one-to-one, condensed for quick lookup. It's here for completeness — the JSON shapes shown in Sections 4-8 are the ones you'll actually build against day to day.

A.1 Meter register-map entry

Field	Type	Meaning
parameter	text	Which measurement this is, e.g. Voltage_L1
modbus_function	text	Read type — almost always "holding register" or "input register"
start_register	number	Address on the meter where this value starts
register_count	number	How many registers this value spans (1, 2, or 4)
data_type	text	Number format: INT16 / UINT16 / INT32 / UINT32 / FLOAT32 / BCD
byte_order	text	Big-endian / little-endian / word-swapped
scale_factor	number	Multiply the raw value by this
offset	number	Add this after scaling
poll_interval_ms	number	How often to read this value, in milliseconds

A.2 Input channel configuration

Field	Type	Meaning
channel_id	number	0-7, which physical terminal
mode	text	ANALOG_0_10V / ANALOG_4_20MA / DIGITAL_10_30V / PULSE
tag_name	text	The internal name this reading is stored under
unit	text	Display unit, e.g. bar, °C, %
calibration.raw_low / value_low	number	The "zero" reference point
calibration.raw_high / value_high	number	The "full-scale" reference point
smoothing_ms	number	Debounce (digital) or averaging window (analog)

A.3 Output configuration

Field	Type	Meaning
output_id	text	Relay_0..Relay_7 or PWM_0..PWM_3

Field	Type	Meaning
label	text	Free-text description shown in the UI
control_mode	text	MANUAL / RULE_DRIVEN / SCHEDULED
linked_rule	text	Rule name, if rule-driven
fail_safe_state	text	State to force on communication loss

A.4 Rule

Field	Type	Meaning
rule_name	text	Human-readable label
watch_tag	text	Which reading this rule monitors
operator	text	GREATER_THAN / LESS_THAN / EQUAL / NOT_EQUAL
threshold	number	Comparison value
hysteresis	number	Buffer before the rule releases
action.output	text	Which output this rule controls
action.type	text	SET_ON / SET_OFF / TOGGLE / SET_LEVEL

> *For the full byte-level wire protocol (frame format, opcodes, flash storage strategy) used by the backend team, see the companion document "SaTTric+ Reconfigurable Rule Engine — Protocol & Firmware Design."*